MENOUFIA UNIVERSITY

Faculty of Computers and Information

SYSTEM ANALYSIS & DESIGN PROJECT

League Fixture Generator System

Item	Details
Project Number	84
Project Name	League Fixture Generator System
Project Type	System Analysis & Design
Academic Year	2027- 2026
Prepared By	Student 1: Omar Asal Student 2: _____
Section	_____
Supervisor	_____

.Prepared according to the supplied System Analysis & Design template

1. Planning (Project Plan)

1.1 Project Identification

Project Details	Field
League Fixture Generator System	Project Name
84	Project Number
Web-based league fixture scheduling and management system	System Type
League Administrator, Team Manager, Viewer	Main Users
Automatically generate valid league fixtures and publish them clearly.	Main Goal

Current Problems

1. Creating fixtures manually takes time and can lead to duplicate or missing matches.
2. Balancing home/away matches manually is difficult.
3. Updating a schedule after postponements can create conflicts.
4. Teams and viewers may not have one reliable, up-to-date schedule.
5. Preparing reports and exporting fixtures manually requires extra effort.

Project Description

The system stores league, season, team, venue and fixture information. The administrator selects a league, registers participating teams, chooses the competition format, and asks the system to generate the fixture list. The system validates the inputs, creates rounds using a round-robin scheduling algorithm, checks for duplicate pairings and conflicts, and then saves and publishes the final schedule. Team managers can view their fixtures, while viewers can browse the published schedule and reports.

2. System Request

Requirement	Category
League Administration / Competition Organizer	Project Sponsor
Reduce manual effort and scheduling errors when creating league fixtures.	Business Need
Provide one centralized source for the current fixture list.	Business Need 2
Make schedule publication faster and easier.	Business Need 3
Manage leagues and seasons; manage teams; generate fixtures; validate conflicts; publish schedules; search/filter fixtures; export reports.	Business Requirements
Time saving, fewer errors, consistent scheduling, faster communication, easier reporting.	Business Value
Correct team data is required; changes after publication require administrator permission; venue/date conflicts must be handled.	Constraints
Users have internet access; the administrator provides valid teams and competition rules.	Assumptions

Success Criteria

- Fixture generation completes quickly for a normal league size.
- No team is scheduled to play two matches in the same round.
- Each required pair of teams is scheduled according to the selected format.
- Published fixtures can be searched and filtered.
- Unauthorized users cannot change league data.

3. Feasibility Analysis

3.1 Technical Feasibility

- The system can be implemented as a web application using HTML/CSS/JavaScript for the interface and a server-side application with a relational database.
- A standard relational DBMS can store teams, seasons, fixtures, venues and users.
- The fixture generator can use a deterministic round-robin algorithm, making the core process technically straightforward.
- The system can be hosted on a local server for the academic prototype and later deployed online.

3.2 Economic Feasibility

- Development can use free/open-source software, reducing software licensing cost.
- The main cost is development time and optional hosting.
- The expected benefit is significant because manual fixture preparation and correction take staff time.
- The project is suitable for a student prototype because it does not require specialized hardware.

3.3 Organizational Feasibility

- The league administrator remains responsible for official data.
- Team managers receive access to view relevant fixtures.
- Users need only short training because the workflow is based on familiar forms and tables.
- The system can fit an existing league workflow without changing the competition rules.

3.4 Schedule Feasibility

The project can be completed as a staged academic project: requirements → analysis → modeling → design → implementation → testing.

4. Time Estimation

Main Output	Estimated Duration	Standard %	Phase
Project plan and system request	4 days	15%	Planning
Requirements, interviews, questionnaire, DFD	6 days	20%	Analysis
ERD, normalization, UML, architecture, interfaces	10 days	35%	Design
Prototype, database, testing and documentation	9 days	30%	Implementation

4.1 Project Task Identification

Est. Time	Resources	Priority	Schedule	Task
6 hours	Word	High	Day 1– 2	System Request
8 hours	Word / spreadsheet	High	Day 2– 4	Feasibility Study
6 hours	Word	Medium	Day 4– 6	Interview
5 hours	Google Forms / spreadsheet	Medium	Day 5– 7	Questionnaire
10 hours	Draw.io / Visio	High	Day 7– 10	DFD
8 hours	Word	High	Day 9– 11	Data Dictionary
8 hours	Word	High	Day 10– 12	Process Specification
12 hours	StarUML / Draw.io	High	Day 12– 16	ERD & UML
7 hours	Word	High	Day 14– 16	Normalization
16 hours	VS Code / Word	High	Day 16– 22	Design
25 hours	VS Code / DBMS	High	Day 22– 31	Implementation & Testing

5. PERT Chart and Gantt Chart

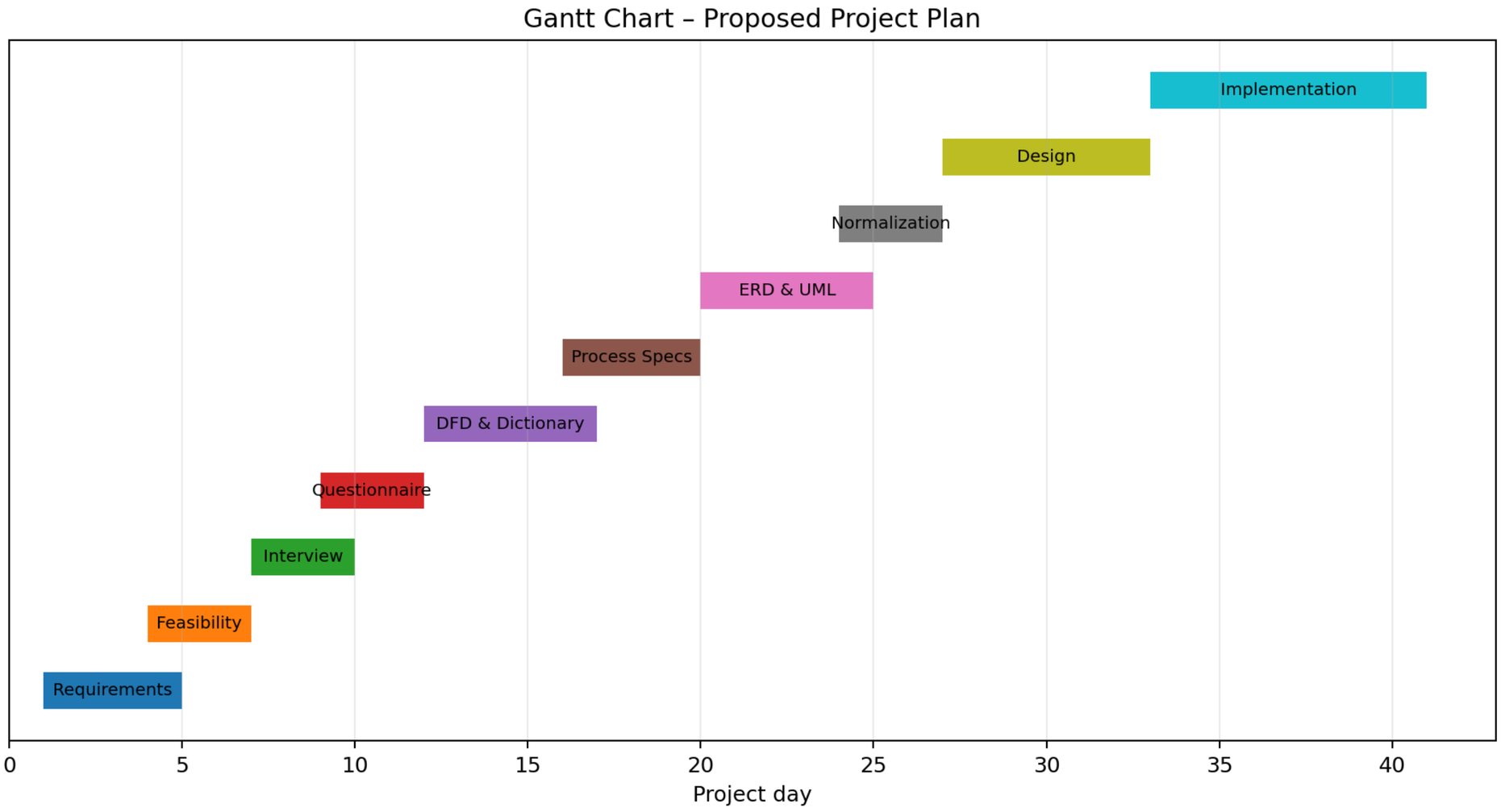


Figure 5.1 — Proposed Gantt chart

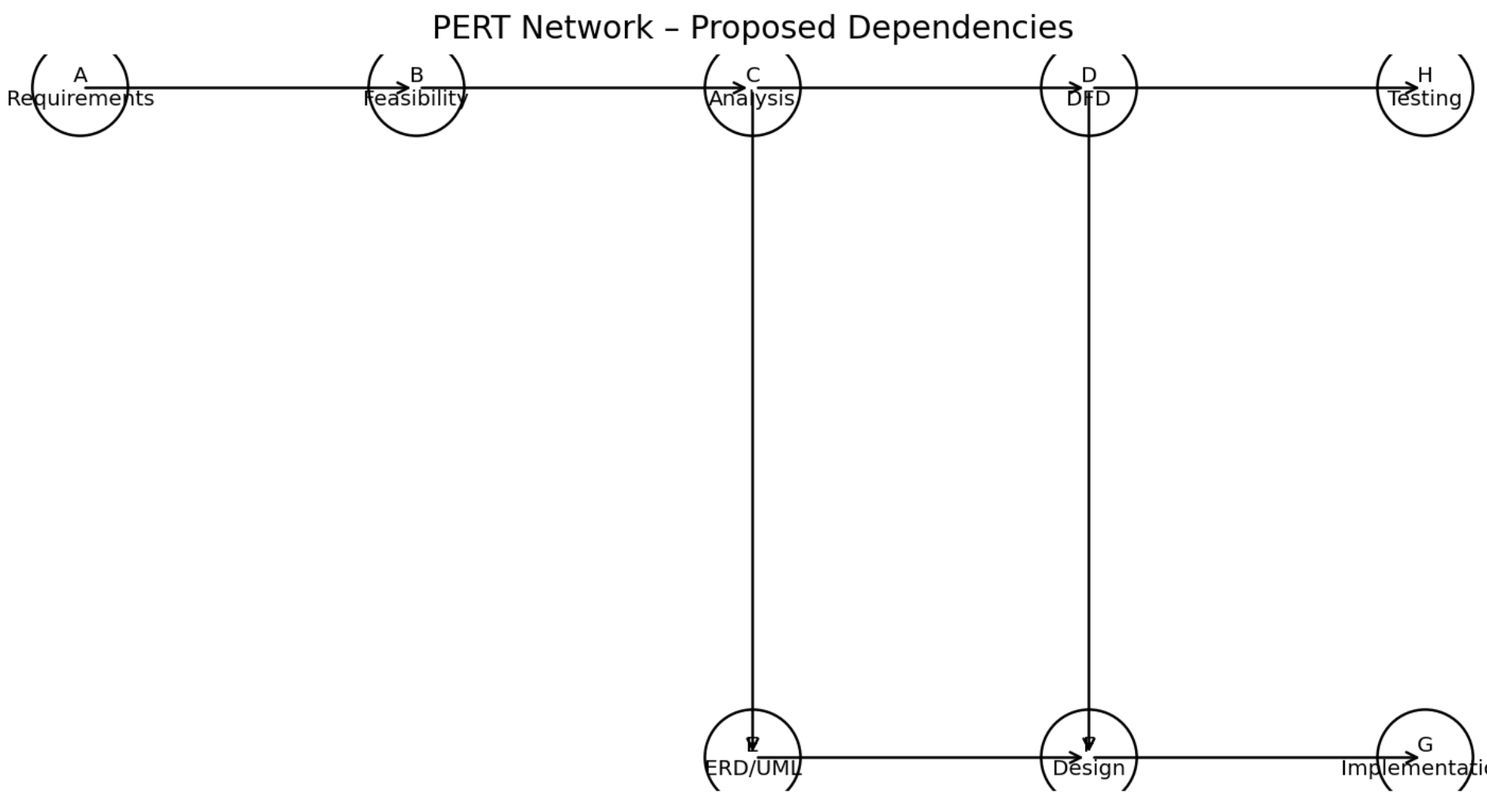


Figure 5.2 — Proposed PERT network

Note: durations are proposed planning estimates for the academic project and can be adjusted to the team's actual schedule.

6. Analysis (Project Proposal)

6.1 Interview

Question	Answer
1. What is the main problem with the current fixture process?	Fixtures are often prepared manually, which takes time and can cause duplicate matches or missed games.
2. Who prepares the official schedule?	The league administrator or competition organizer.
3. What information is needed before generating fixtures?	League name, season, teams, competition format, and optional venue/date constraints.
4. How should the system generate fixtures?	It should generate a round-robin schedule according to the selected format.
5. What are common errors?	Duplicate pairings, two matches for one team in the same round, and incorrect dates.
6. What are the critical objectives?	Correct fixture generation, easy updates, fast publication, and clear access for viewers.
7. What technology is suitable?	A web interface connected to a relational database and a scheduling service.
8. Should users be able to edit published fixtures?	Only authorized administrators should edit official fixtures.
9. What reports are useful?	Full league schedule, team-specific fixtures, round list, and postponed/cancelled matches.
10. Is there anything else to add?	The system should keep a history of fixture changes and provide clear status information.

7. Interview Reports

Interview Report — Competition Organizer

Role: Primary stakeholder

Summary: Needs a reliable way to generate and publish the official fixture list.

- Manage seasons and teams.
- Generate fixtures automatically.
- Approve and publish the schedule.
- Export reports.

Interview Report — Team Manager

Role: Secondary stakeholder

Summary: Needs quick access to the team's matches and updates.

- View team fixtures.
- Filter by round/date.
- Receive updated schedule information.
- Cannot alter official fixtures.

Interview Report — Viewer / Fan

Role: End user

Summary: Needs simple public access to the current schedule.

- Browse fixtures.
- Filter by team or round.
- See match status.
- View published updates.

8. Questionnaire & Responses

The questionnaire below is designed for league administrators, team managers and viewers. The statistics in the next section are a proposed sample dataset for documentation; replace them with actual Google Forms results before final submission if required.

No.	Question	Response Type
1	Do you currently find manual fixture creation time-consuming?	Yes / No
2	Have you experienced duplicate or missing fixtures?	Yes / No
3	Would automatic fixture generation reduce your work?	Yes / No
4	Is team-specific fixture search important?	Yes / No
5	Is it important to prevent two matches for one team in the same round?	Yes / No
6	Would you use a web-based published schedule?	Yes / No
7	Is filtering by date/round/team useful?	Yes / No
8	Should only administrators edit fixtures?	Yes / No
9	Would export to PDF/Excel be useful?	Yes / No
10	Do you want visible status for postponed/cancelled matches?	Yes / No

9. Questionnaire Statistics

Percentage	Yes (sample)	Topic	Q
80%	24/30	Manual fixture creation is time-consuming	1
60%	18/30	Experienced duplicate/missing fixtures	2
93%	28/30	Automatic generation would reduce work	3
90%	27/30	Team-specific search is important	4
97%	29/30	Conflict prevention is important	5
87%	26/30	Would use web-based schedule	6
83%	25/30	Filtering is useful	7
97%	29/30	Only admins should edit	8
77%	23/30	PDF/Excel export is useful	9
93%	28/30	Status for postponed/cancelled matches is useful	10

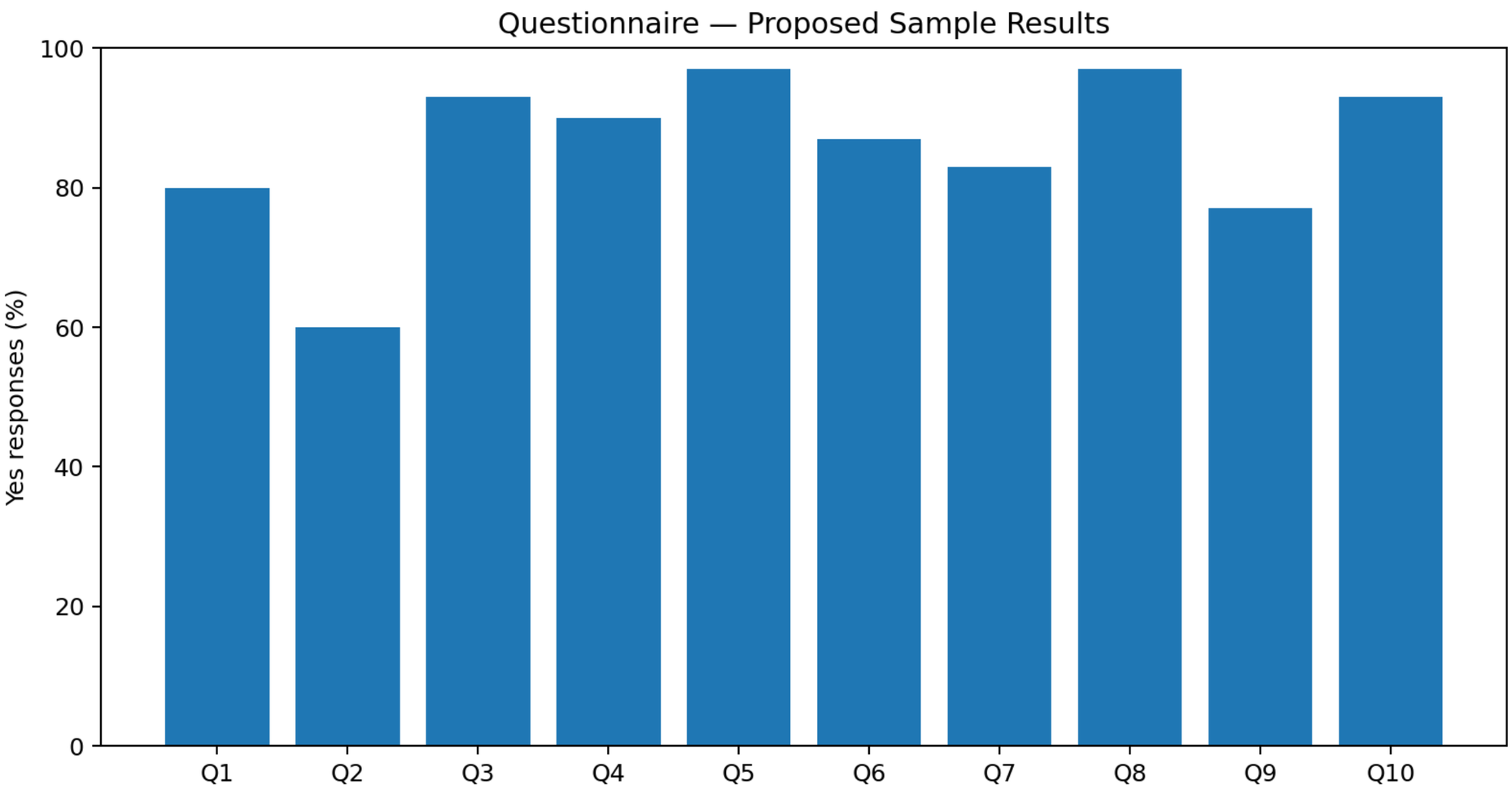


Figure 9.1 — Proposed questionnaire results (30 sample responses)

SYSTEM ANALYSIS & DESIGN — League Fixture Generator System

10. Data Flow Diagram — Context Diagram

The context diagram treats the League Fixture Generator System as one process and shows its main external entities and information exchanges.

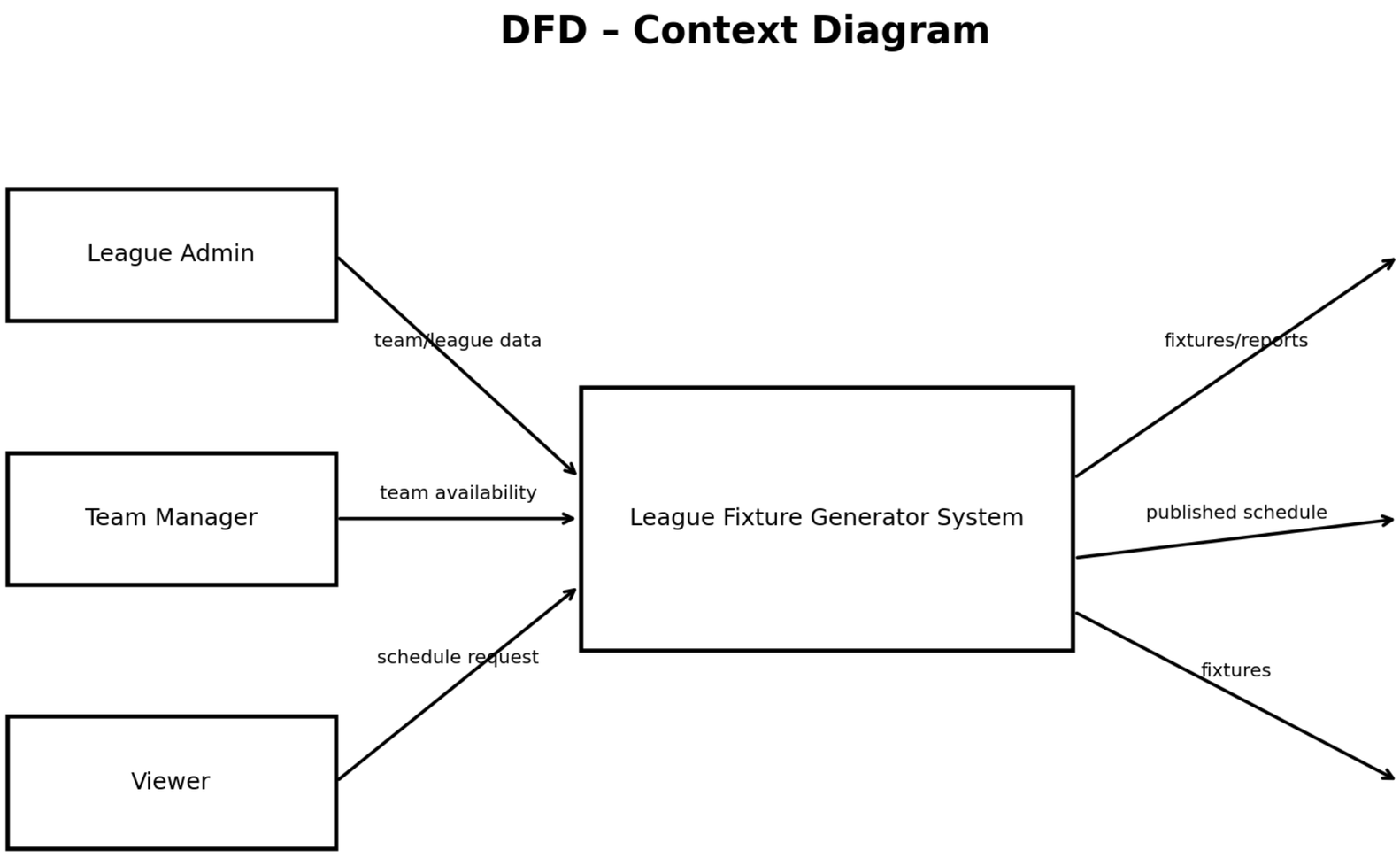


Figure 10.1 — Context DFD

Output	Input	External Entity
Generated schedule, validation messages, reports	League/team data, generation request, update request	League Admin
Team fixtures, update notifications	Team information / availability	Team Manager
Published fixtures and statuses	Search/filter request	Viewer

11. Data Flow Diagram — Level 0

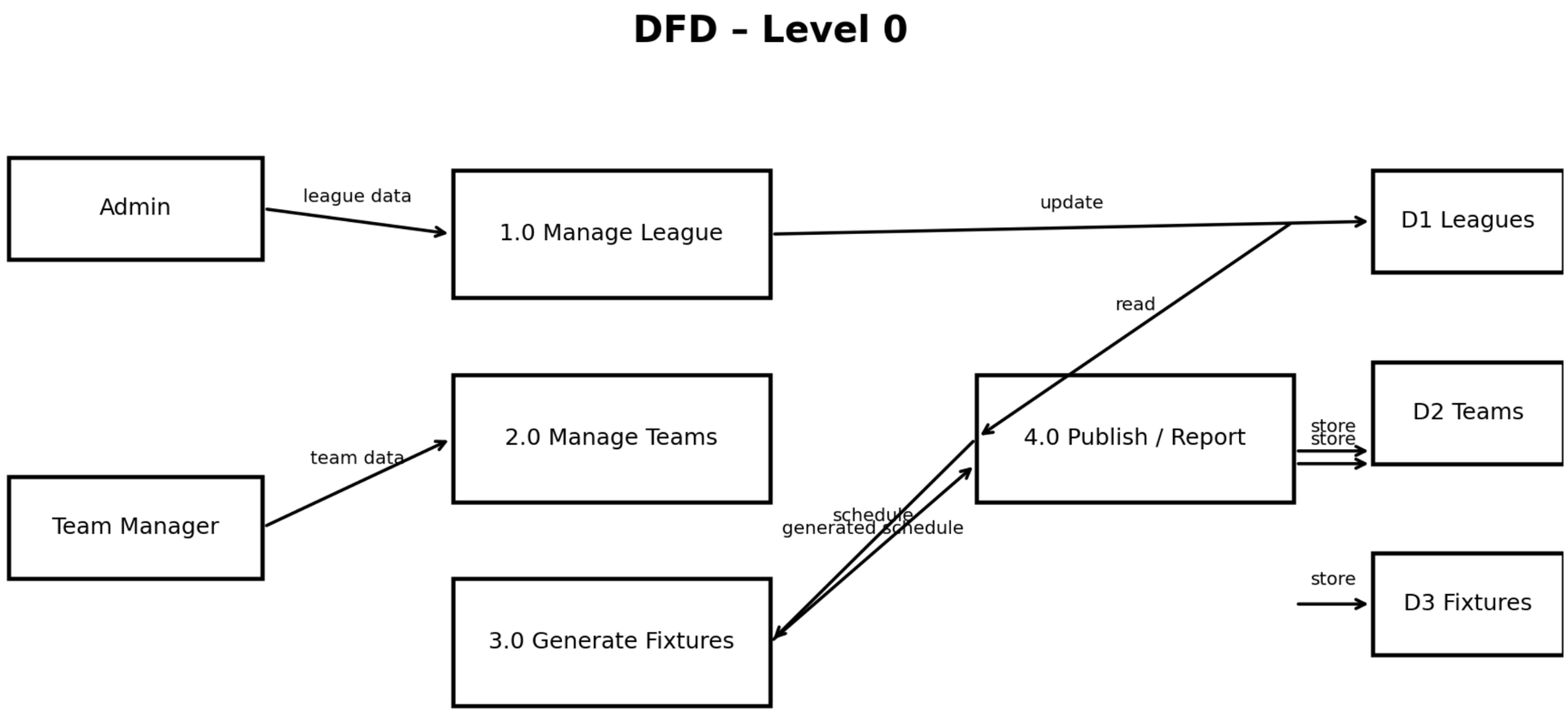


Figure 11.1 — Level 0 DFD

Process	Description
1.0 Manage League	Create/update league and season information.
2.0 Manage Teams	Register teams and maintain team information.
3.0 Generate Fixtures	Validate teams, select format, generate rounds and check conflicts.
4.0 Publish / Report	Publish approved fixtures and generate reports.

12. Data Flow Diagram — Level 1

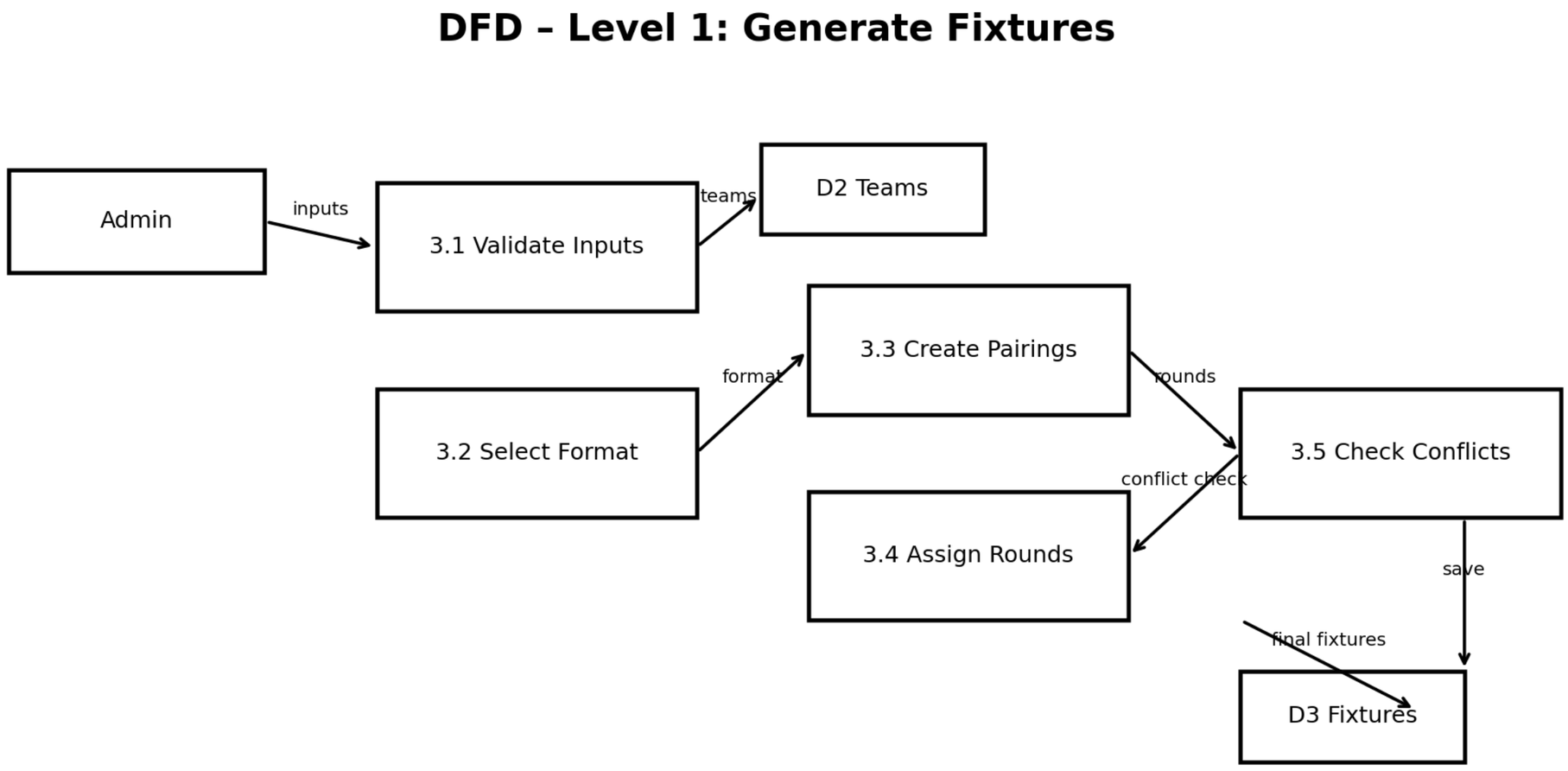


Figure 12.1 — Level 1 for Process 3.0 Generate Fixtures

Validate the number and status of participating teams. .6

Read the selected competition format. .7

Generate pairings using round-robin logic. .8

Assign matches to rounds and dates where possible. .9

Check for conflicts and duplicate pairings. .10

Save the final fixture set and return the result to the administrator. .11

13. Data Dictionary

13.1 Data Flow Description

Description	Source → Destination	Name	ID
League name, format, season	League Admin → Manage League	League Data	F1
Team ID, name, city, status	Team Manager/Admin → Manage Teams	Team Data	F2
League, season, format, constraints	Admin → Generate Fixtures	Generation Request	F3
Fixture ID, teams, round, date, venue, status	Generate Fixtures → Publish/Report	Generated Fixtures	F4
Official fixture information	Publish/Report → Viewer/Team Manager	Published Schedule	F5
Validation/conflict details	Generate Fixtures → Admin	Conflict Message	F6
Report type and filters	Admin → Publish/Report	Report Request	F7
Filtered/exportable schedule	Publish/Report → Admin	Schedule Report	F8

14. Data Structures for Data Flow

Elements	Structure
LeagueID + Name + Format + SeasonID + Status	LeagueData
TeamID + TeamName + City + Contact + Status	TeamData
LeagueID + SeasonID + Format + StartDate + Frequency + Constraints	GenerationRequest
FixtureID + SeasonID + RoundNo + MatchDate + HomeTeamID + AwayTeamID + VenueID + Status	FixtureData
SeasonID + RoundNo + FixtureData + PublicationDate	PublishedSchedule
ReportID + ReportType + Filters + GeneratedDate + CreatedBy	ReportData

14.1 Structural Record

A structural record groups related elements into a single logical record. For example, FixtureData represents one scheduled match and can be stored as one database record.

15. Data Elements

Base/Derived	Length/Format	Type	Description	Name
Base	10 digits	Integer	Unique league identifier	LeagueID
Base	100 chars	Text	Official league name	LeagueName
Base	10 digits	Integer	Unique season identifier	SeasonID
Base	10 digits	Integer	Unique team identifier	TeamID
Base	100 chars	Text	Team name	TeamName
Base	20 chars	Text	Competition format	Format
Base	10 digits	Integer	Unique fixture identifier	FixtureID
Derived/Base	3 digits	Integer	Round number	RoundNo
Base	Standard date-time	DateTime	Scheduled match date/time	MatchDate
Base	10 digits	Integer	Venue identifier	VenueID
Base	20 chars	Text	Scheduled/Postponed/Cancelled/Completed	Status
Base	10 digits	Integer	User/admin identifier	CreatedBy

16. Data Stores

ID	Name	Type	Data	Primary Key	Comments
D1	LeagueStore	Database	League and season records	LeagueID / SeasonID	Stores league setup information.
D2	TeamStore	Database	Team records	TeamID	Stores participating teams.
D3	FixtureStore	Database	Fixture records	FixtureID	Stores generated and published matches.
D4	VenueStore	Database	Venue records	VenueID	Stores optional venue details.
D5	UserStore	Database	User/admin records	UserID	Stores authenticated users and roles.
D6	AuditStore	Database	Change history	AuditID	Stores important schedule changes.

17. Process Specification

17.1 Process 1.0 — Manage League

Input: LeagueData. Output: stored/updated league record.

Process Logic:

Receive league details. .12
Validate required fields. .13
Create or update league and season records. .14
Return success or validation message. .15

17.2 Process 2.0 — Manage Teams

Input: TeamData. Output: updated TeamStore.

Receive team information. .16
Check unique TeamID/name rules. .17
Save active team record. .18
Return confirmation. .19

17.3 Process 3.0 — Generate Fixtures

Input: GenerationRequest + TeamStore. Output: FixtureData or conflict message.

IF number of active teams is less than the minimum required, return an error. .20
IF the number of teams is odd, add a BYE placeholder for single round-robin generation. .21
Generate pairings using the selected round-robin method. .22
Assign each pairing to a round. .23
Validate that each team appears at most once per round. .24
Validate duplicate pairings and apply home/away rules where applicable. .25
Save valid fixtures. .26

18. Process Specification — Continued

18.1 Process 4.0 — Publish / Report

Input: FixtureData. Output: PublishedSchedule + ReportData.

Retrieve approved fixtures. .27

Apply selected filters such as team, round, date or status. .28

Generate the display/report format. .29

Publish the official schedule. .30

Store the publication/change event in the audit log. .31

18.2 Decision Table — Fixture Validation

Case 4	Case 3	Case 2	Case 1	Condition / Action
N	N	Y	Y	Valid team count
-	-	N	Y	No duplicate pairings
-	-	-	Y	No same-round conflict
Return validation error	Return team-count error	Return duplicate error	Save fixtures	Action

19. Normalization

19.1 Unnormalized Relation

A single wide record might contain repeated team, season and fixture information. Example:

Status	Venue	Date	Round	Team2	Team1	Season	LeagueName	LeagueID
Scheduled	Main Stadium	2026-09-01	1	Lions	Tigers	2026	Premier League	L01
Scheduled	City Stadium	2026-09-01	1	Sharks	Eagles	2026	Premier League	L01
Scheduled	Main Stadium	2026-09-08	2	Eagles	Tigers	2026	Premier League	L01

19.2 First Normal Form (1NF)

All fields are atomic and each row represents one fixture. Repeating groups are removed.

Status	VenueID	MatchDate	RoundNo	AwayTeamID	HomeTeamID	SeasonID	LeagueID	FixtureID
Scheduled	V01	2026-09-01	1	T02	T01	S01	L01	F001
Scheduled	V02	2026-09-01	1	T04	T03	S01	L01	F002

20. Normalization — 2NF and 3NF

20.1 Second Normal Form (2NF)

Partial dependencies are removed by separating league, season, team, venue and fixture data. Fixture-specific attributes remain dependent on FixtureID.

Main Attributes	Key	Table
LeagueName, Format	LeagueID	League
LeagueID, SeasonYear, Status	SeasonID	Season
TeamName, City, Status	TeamID	Team
VenueName, Location	VenueID	Venue
SeasonID, HomeTeamID, AwayTeamID, RoundNo, MatchDate, VenueID, Status	FixtureID	Fixture

20.2 Third Normal Form (3NF)

Transitive dependencies are removed. For example, LeagueName is stored only in League, and TeamName is stored only in Team. Fixture stores foreign keys rather than repeating descriptive data.

Foreign Keys	Primary Key	3NF Table
—	LeagueID	League
LeagueID	SeasonID	Season
—	TeamID	Team
—	VenueID	Venue
SeasonID, HomeTeamID, AwayTeamID, VenueID	FixtureID	Fixture
—	UserID	User
UserID, FixtureID	AuditID	AuditLog

21. Entity Relationship Diagram (ERD)

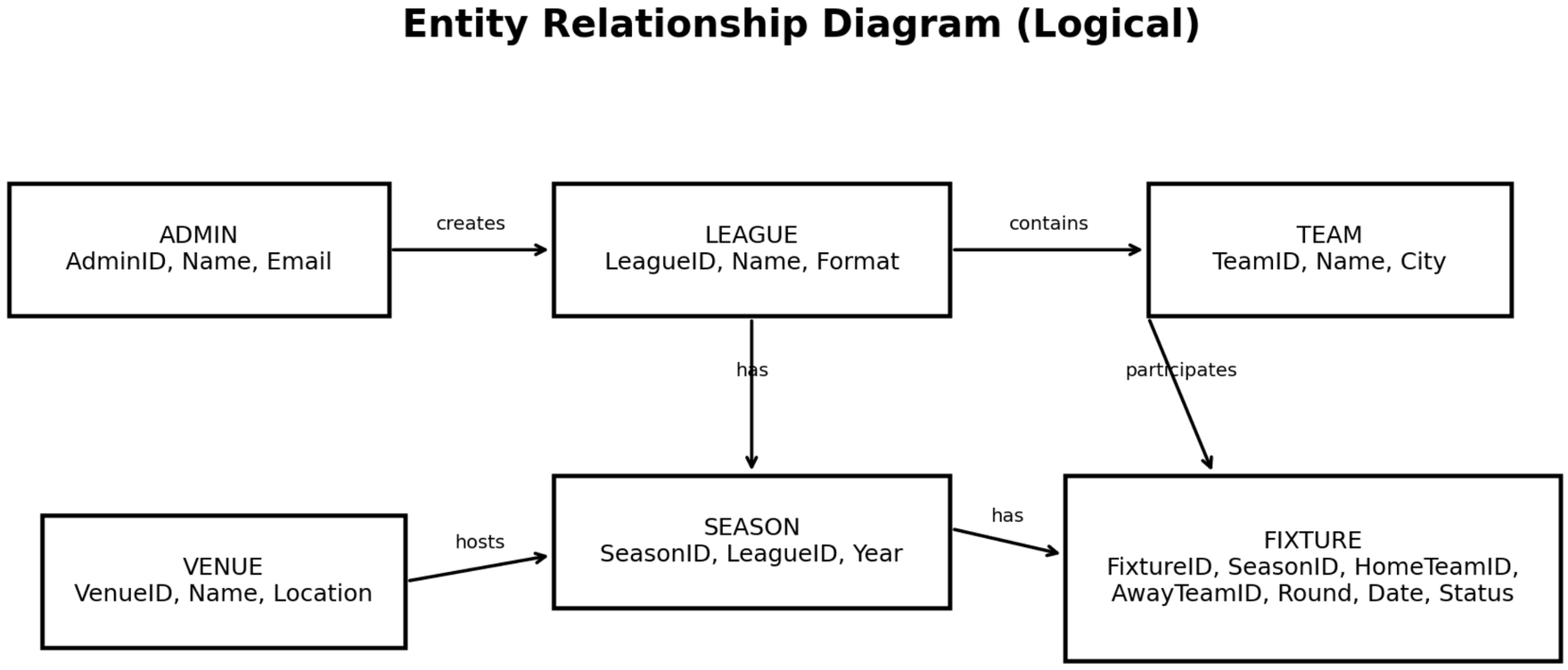


Figure 21.1 — Logical ERD

21.1 Main Relationships

- One League can have many Seasons.
- One Season can contain many Fixtures.
- A Team can appear in many Fixtures as home team and many Fixtures as away team.
- One Venue can host many Fixtures.
- One Admin/User can create or publish many fixtures.
- Audit records preserve important changes to official fixtures.

22. UML – Use Case Diagram

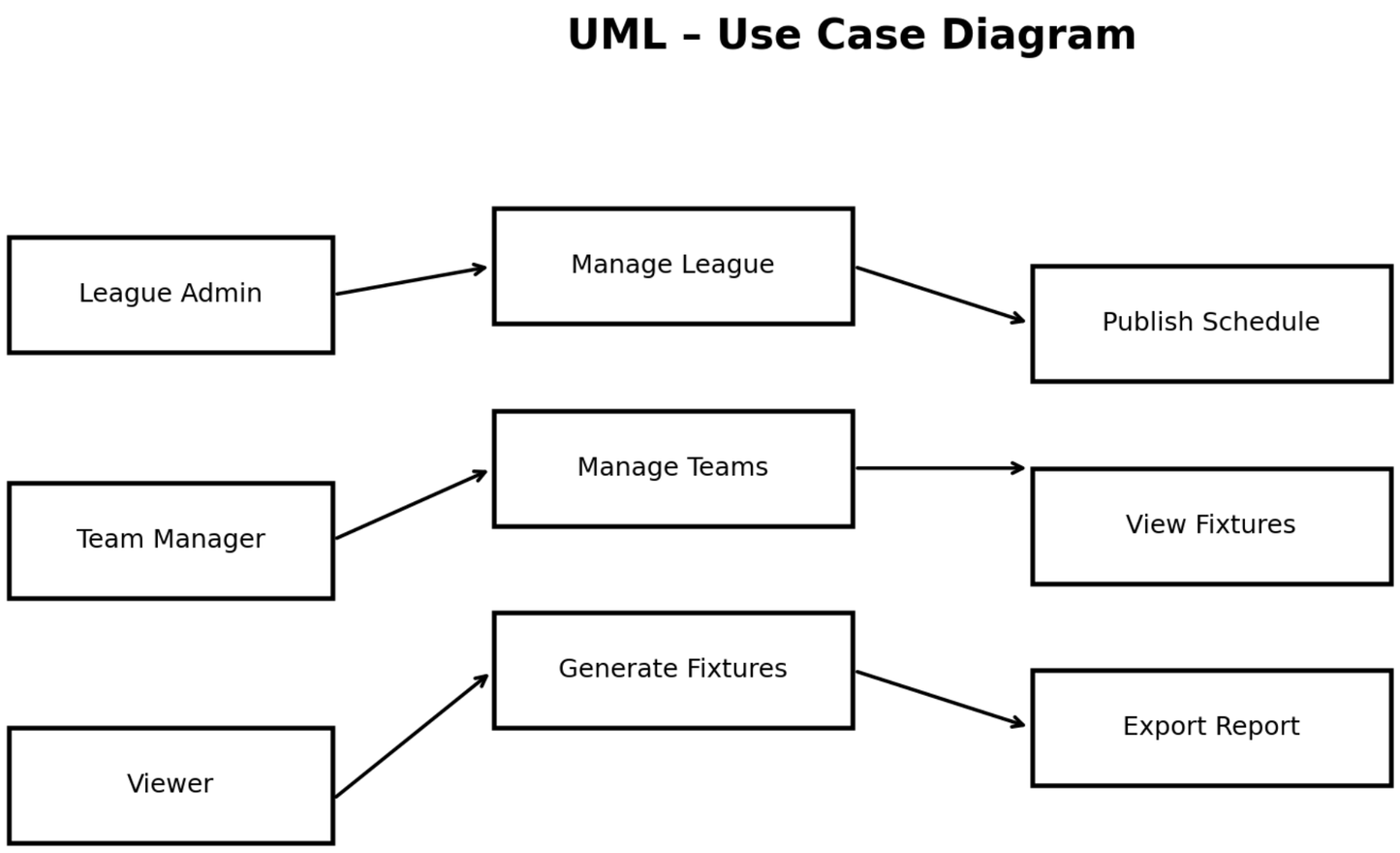


Figure 22.1 — Use Case Diagram

Use Cases	Actor
Login, Manage League, Manage Teams, Generate Fixtures, Validate Conflicts, Publish Schedule, Export Reports	League Admin
Login, View Team Fixtures, View Updates	Team Manager
View Published Fixtures, Search/Filter Schedule	Viewer

23. UML – Class Diagram

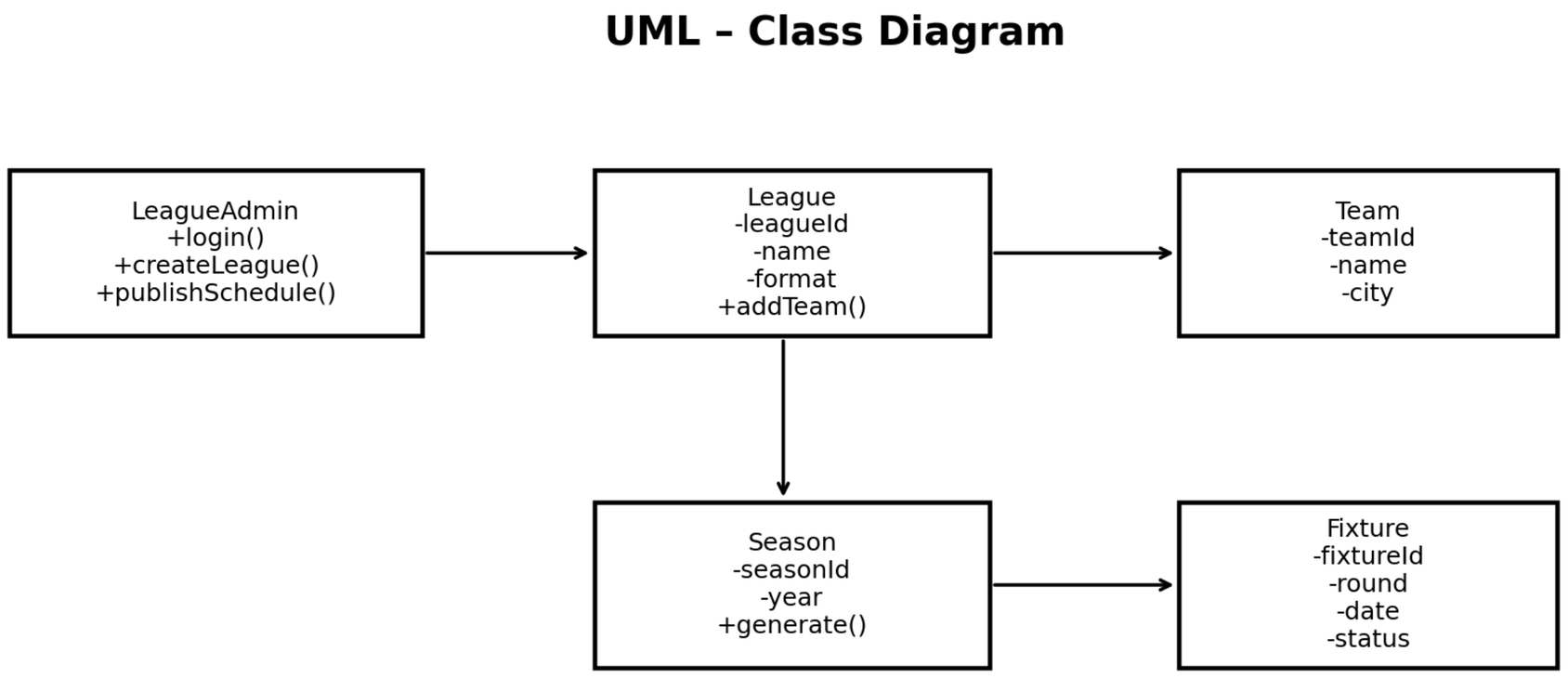


Figure 23.1 — Class Diagram

Class	Important Attributes	Important Methods
League	leagueId, name, format	addTeam(), createSeason()
Season	seasonId, year, status	generateFixtures(), publish()
Team	teamId, name, city	getFixtures()
Fixture	fixtureId, round, date, status	validate(), updateStatus()
LeagueAdmin	adminId, name, email	login(), manageLeague(), publishSchedule()

24. UML – Sequence Diagram

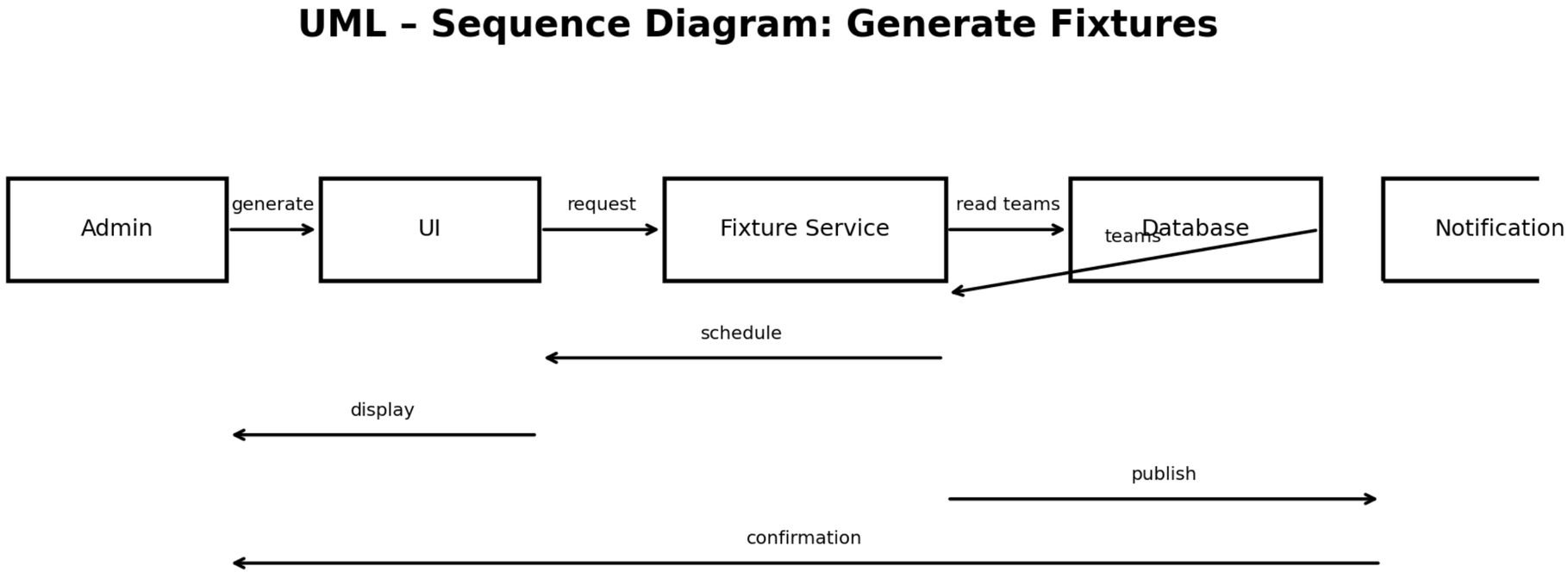


Figure 24.1 — Sequence Diagram: Generate Fixtures

Administrator chooses a league/season and clicks Generate Fixtures. .32
UI sends a generation request to the Fixture Service. .33
Fixture Service reads active teams from the database. .34
The service creates pairings and assigns rounds. .35
The service checks conflicts and duplicate pairings. .36
Valid fixtures are saved to the database. .37
The UI displays the generated schedule and confirmation. .38

25. Design (Project Specification)

25.1 Architecture Design — Operational Requirements

Specification	Requirement
Web browser, application server, relational database.	Technical Environment
Application layer communicates with database through secure APIs/services.	System Integration
Responsive interface usable on desktop, tablet and mobile.	Portability
Layered structure: UI, business logic, data access.	Maintainability
Regular database backups and audit history.	Backup

25.2 Performance Requirements

Target	Requirement
Normal page actions should respond within a few seconds in the academic environment.	Speed
Support typical league sizes such as 8– 32 teams.	Capacity
System should remain available during schedule publication periods.	Availability
Generated fixture set must pass validation before publication.	Reliability

25.3 Security Requirements

Specification	Requirement
Role-based permissions.	Access Control
Secure login for administrators/team managers.	Authentication
HTTPS in deployment; passwords stored as hashes.	Encryption
Record important schedule changes.	Audit
Server-side validation, file restrictions and regular backups.	Virus/Abuse Control

26. Data Storage Design

26.1 Data Storage Format

Relational database. Suggested tables: Users, Leagues, Seasons, Teams, Venues, Fixtures, AuditLogs.

26.2 Physical Table Specification

Fields	Key	Table
Name VARCHAR(100), Format VARCHAR(20), Status VARCHAR(20)	LeagueID INT PK	Leagues
LeagueID INT FK, SeasonYear INT, Status VARCHAR(20)	SeasonID INT PK	Seasons
Name VARCHAR(100), City VARCHAR(100), Status VARCHAR(20)	TeamID INT PK	Teams
Name VARCHAR(100), Location VARCHAR(150)	VenueID INT PK	Venues
SeasonID INT FK, HomeTeamID INT FK, AwayTeamID INT FK, RoundNo INT, MatchDate DATETIME, VenueID INT FK, Status VARCHAR(20)	FixtureID INT PK	Fixtures
Name VARCHAR(100), Email VARCHAR(150), PasswordHash VARCHAR(255), Role VARCHAR(30)	UserID INT PK	Users
UserID INT FK, FixtureID INT FK, Action VARCHAR(50), ActionDate DATETIME	AuditID INT PK	AuditLogs

27. Convert Logical ERD to Physical ERD

The logical entities are converted to database tables with primary keys and foreign keys. Referential integrity is enforced between Seasons- Leagues, Fixtures- Seasons, Fixtures- Teams and Fixtures- Venues.

Foreign Key	Cardinality	Relationship
Season.LeagueID	1 : M	League → Season
Fixture.SeasonID	1 : M	Season → Fixture
Fixture.HomeTeamID	1 : M	Team → Fixture (Home)
Fixture.AwayTeamID	1 : M	Team → Fixture (Away)
Fixture.VenueID	1 : M	Venue → Fixture
AuditLog.UserID	1 : M	User → AuditLog

28. Program Design

28.1 Convert Logical DFD to Physical DFD

- Browser UI sends authenticated requests to the application server.
- Application server validates input and applies scheduling rules.
- Fixture Generator Service runs the round-robin algorithm.
- Database stores leagues, teams, seasons, fixtures and audit logs.
- Reporting service formats schedule data for screen and export.

28.2 Core Fixture Generation Logic

For an even number of teams, pair teams round by round using a round-robin rotation. For an odd number, add a BYE placeholder. After pair generation, validate that each team appears once per round and that every required pairing occurs the required number of times.

28.3 Validation Rules

Team IDs must be unique. .39

League and season must exist before fixture generation. .40

Only active teams participate. .41

HomeTeamID and AwayTeamID cannot be equal. .42

One team cannot have two fixtures in the same round. .43

Published fixtures can only be changed by authorized roles. .44

Every fixture must have a valid status. .45

29. Interface Design

29.1 Use Scenario Development

Steps	Scenario
Admin logs in → selects league/season → selects format → clicks Generate → reviews validation → saves → publishes.	Generate Fixtures
Admin opens Teams → adds/edits team → validates unique name/ID → saves.	Manage Teams
Team Manager logs in → selects team → filters season → views matches.	View Team Fixtures
Admin reviews generated fixtures → confirms → publishes → system records audit event.	Publish Schedule
Admin selects filters → chooses PDF/Excel → system generates report.	Export Report

29.2 Interface Structure

- Login Page
- Dashboard
- League & Season Management
- Team Management
- Fixture Generator
- Fixture Review / Conflict Panel
- Published Schedule
- Reports & Export
- User / Role Management
- Audit Log

30. Interface Standards Design

30.1 Interface Metaphor

The interface uses a sports-league dashboard metaphor: teams, rounds, fixtures and statuses are presented as familiar cards, tables and filters.

30.2 Interface Template

Main Controls	Screen
League selector, season selector, summary cards, latest updates	Dashboard
Team list, format selector, start date, generate button	Fixture Generator
Round tabs, conflict warnings, save/publish controls	Fixture Review
Search, team filter, round filter, date filter	Schedule
Report type, date range, export buttons	Reports

30.3 Interface Design Rules

- Use consistent navigation and labels.
- Show validation errors beside the relevant field.
- Require confirmation before publishing or deleting official data.
- Use clear status labels: Scheduled, Postponed, Cancelled, Completed.
- Keep tables readable on mobile through responsive layout.

31. Interface Design Prototyping

31.1 Dashboard Prototype

Prototype Content	Area
League Fixture Generator | User | Logout	Header
Teams: 16 | Fixtures: 240 | Next Round: 8 | Conflicts: 0	Summary Cards
Manage Teams | Generate Fixtures | Publish Schedule | Reports	Quick Actions
Round, Date, Home Team, Away Team, Venue, Status	Upcoming Fixtures
Postponed matches, conflicts and unpublished changes	Alerts

31.2 Fixture Generator Prototype

Example	Control
Premier League	League
2026/2027	Season
Double Round-Robin	Format
01/09/2026	Start Date
Weekly	Match Frequency
[Generate Fixtures]	Generate
No duplicate pairings / No same-round conflict	Validation

32. Implementation (New System)

32.1 Suggested Technology Stack

Suggested Technology	Layer
HTML5, CSS3, JavaScript	Frontend
Java Spring Boot or Node.js/Express	Backend
MySQL or PostgreSQL	Database
Draw.io / StarUML	Modeling
Visual Studio Code	Development
Git / GitHub	Version Control

32.2 Implementation Modules

Authentication and role management. .46
League and season management. .47
Team management. .48
Fixture generation engine. .49
Conflict validation. .50
Fixture publication. .51
Search and filtering. .52
Reports/export. .53
Audit logging. .54

32.3 Testing Plan

Expected Result	Test Case	Test ID
League saved successfully	Create league with valid data	T01
Validation error	Add duplicate team	T02
Valid round-robin schedule generated	Generate fixtures for 8 teams	T03
System rejects conflict	Check same team twice in a round	T04
Schedule becomes visible to viewers	Publish valid schedule	T05
Access denied	Unauthorized edit	T06
Only selected team's fixtures shown	Filter by team	T07
Correct filtered report generated	Export report	T08

33. Implementation — Database Example

The following SQL-style definitions show the intended physical structure. They are documentation examples and can be adapted to the selected DBMS.

```
CREATE TABLE Teams (
    TeamID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL UNIQUE,
    City VARCHAR(100),
    Status VARCHAR(20) NOT NULL
);

CREATE TABLE Seasons (
    SeasonID INT PRIMARY KEY,
    LeagueID INT NOT NULL,
    SeasonYear INT NOT NULL,
    Status VARCHAR(20) NOT NULL
);

CREATE TABLE Fixtures (
    FixtureID INT PRIMARY KEY,
    SeasonID INT NOT NULL,
    HomeTeamID INT NOT NULL,
    AwayTeamID INT NOT NULL,
    RoundNo INT NOT NULL,
    MatchDate DATETIME,
    VenueID INT,
    Status VARCHAR(20) NOT NULL,
    FOREIGN KEY (SeasonID) REFERENCES Seasons(SeasonID),
    FOREIGN KEY (HomeTeamID) REFERENCES Teams(TeamID),
    FOREIGN KEY (AwayTeamID) REFERENCES Teams(TeamID)
);)
```

Application logic must additionally enforce HomeTeamID <> AwayTeamID and the one-match-per-team-per-round rule.

34. End-to-End System Workflow

Administrator logs in. .55
Administrator creates or selects a league. .56
Administrator creates/selects a season. .57
Administrator adds participating teams. .58
Administrator selects competition format. .59
System validates the number and status of teams. .60
Fixture engine generates pairings and rounds. .61
System checks duplicate and same-round conflicts. .62
Administrator reviews generated fixtures. .63
Administrator saves and publishes the official schedule. .64
Team managers and viewers access the published schedule. .65
Changes such as postponements are recorded and republished. .66
Reports can be filtered and exported. .67

34.1 Benefits of the New System

- Faster fixture generation.
- Lower risk of manual scheduling errors.
- Centralized and consistent schedule.
- Easy team-specific search.
- Controlled publishing and auditing.
- Reusable for different leagues and seasons.

35. Conclusion

The League Fixture Generator System provides a structured solution to the main problems associated with manual league scheduling. The analysis identifies the stakeholders, requirements, data flows and business rules. The design defines the database, ERD, DFDs, UML models, architecture and interface structure. The proposed implementation uses a relational database and a fixture-generation service to create, validate and publish schedules.

35.1 Future Enhancements

- Automatic SMS/email notifications for schedule changes.
- Venue availability integration.
- Referee assignment.
- Live match results and standings.
- Mobile application.
- Advanced optimization for travel distance and home/away balance.
- Public API for external sports applications.

35.2 Submission Checklist

Item	Status
Project Identification	Completed
System Request	Completed
Feasibility Analysis	Completed
Time Estimation / Tasks	Completed
PERT / Gantt	Completed
Interview	Completed
Questionnaire + sample statistics	Completed — replace sample stats with real responses if required
Context DFD / Level 0 / Level 1	Completed
Data Dictionary	Completed
Process Specification	Completed
Normalization 1NF / 2NF / 3NF	Completed
ERD / Use Case / Class / Sequence	Completed
Architecture / Storage / Program Design	Completed
Interface Design / Prototype	Completed
Implementation / Testing	Completed

SYSTEM ANALYSIS & DESIGN — League Fixture Generator System

End of Project Documentation